

Indian Institute of Engineering Science and Technology, Shibpur
Department of Computer Science and Technology

Algorithm Lab (CS2271)
Assignment 3

1. In an online examination platform, the examination scores of 100,000 students are stored sequentially in an array on the server. Once the examination process is completed, the examination authority needs to sort the students' scores efficiently in order to prepare rank lists and perform further analysis. Write a C program to implement the Randomized Quicksort algorithm for sorting the array of students' scores. The program should consider and handle the following three input scenarios:
 - i) Best-case-like scenario: when the array of scores is already sorted in ascending order.
 - ii) Worst-case-like scenario for deterministic approaches: when the array is sorted in reverse (descending) order.
 - iii) Average-case scenario: when the array elements are in a random, unsorted order.

Finally, count the total number of comparisons performed during the sorting process and measure the execution time required to complete the sorting operation.

2. Write a C program to study the impact of duplicate keys on the performance of standard quicksort and randomized quicksort. The program should generate three integer arrays, each of size 100,000, with varying proportions of duplicate elements in order to simulate different real-world data distributions. The arrays should be constructed as follows:
 - i) Array-I: Contains approximately 90% duplicate values, representing highly repetitive data.
 - ii) Array-II: Contains approximately 50% duplicate values, representing moderately repetitive data.
 - iii) Array-III: Contains approximately 10% duplicate values, representing data with low repetition.

After generating the three arrays, implement and apply the Standard Quicksort (a fixed pivot selection strategy) and Randomized Quicksort (a random pivot selection strategy). For each combination of array type and sorting algorithm, the program must count the total number of key comparisons performed during sorting, measure the maximum recursion depth encountered during execution, and measure the execution time required to complete the sorting process.

Finally, analyze and compare the performance of Standard Quicksort and Randomized Quicksort across all three data distributions. Plot appropriate 2D graphs (such as input distribution vs. execution time, number of comparisons, and recursion depth) to visually illustrate the effect of duplicate values on the efficiency and stability of both algorithms.

3. Consider an integer array of size 100,000 in which approximately 80% of the elements are already in nearly sorted order, while the remaining elements are randomly positioned. Such data distributions are common in real-world applications.

Write a C program to implement both the Quicksort and Merge Sort algorithms for sorting the given array. For both sorting technique, Count the total number of key comparisons performed during execution, measure the maximum recursion depth reached during the sorting process, measure and record the execution time required to complete the sorting operation. Based on a comparative analysis of these performance metrics, justify which algorithm is more suitable for the given input conditions.